THEME: VIS + AI · INFORMATION VISUALIZATION TERM PROJECT PROPOSAL

# LagLens: What Does a Robot Do While Its Policy Is Still Thinking?

A visual analytics tool for robot policies that plan in chunks and need time to think

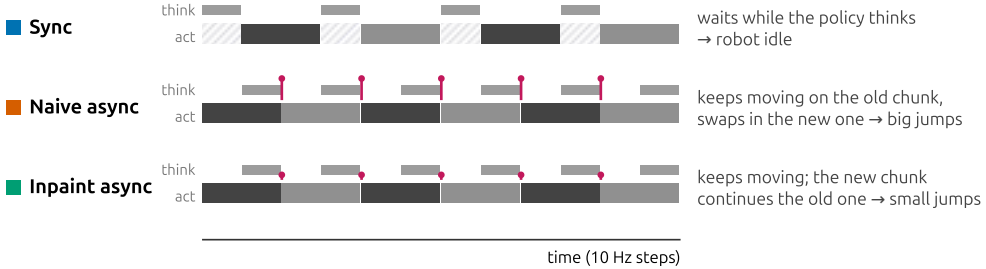
## How such a robot works

1. **Plan.** A robot **policy** (a neural network) looks at the camera once and plans its next 16 moves: an **action chunk**.
2. **Act.** The robot performs the first *h* moves of the chunk without looking again, then needs a new chunk.
3. **Think.** Computing a chunk takes time *d* (**inference latency**, here 0–600 ms). What should the robot do meanwhile? (right)
   • **Sync**: **wait**. Safe, but the robot stands idle.
   • **Async**: **keep moving** on the old chunk. No waiting, but the scene changes while the policy thinks, so the new chunk is made for a scene that no longer exists (an **outdated plan**) and may not continue the current motion: the robot **jumps** when it switches (a **chunk-boundary discontinuity**). **Naive async** just swaps the chunks; **inpaint async** computes the new chunk so that it continues the old one.

**The problem.** Engineers choose the strategy, *d* and *h* from one success rate per setting, e.g. “naive async, d = 4: 62 % vs. sync, d = 0: 72 %”. That number cannot show **which** situations broke, **when**, or **why** (idling, outdated plans or jumps). We found no visualization tool for this.

**Our idea: same start, different schedule.** We run one pretrained policy from the **same 50 start states** under **46 schedules** (strategy × *d* × *h*) and record every planned chunk. Any two schedules can then be compared start by start and replayed move by move.

## The three strategies over time (d = 2, h = 4)



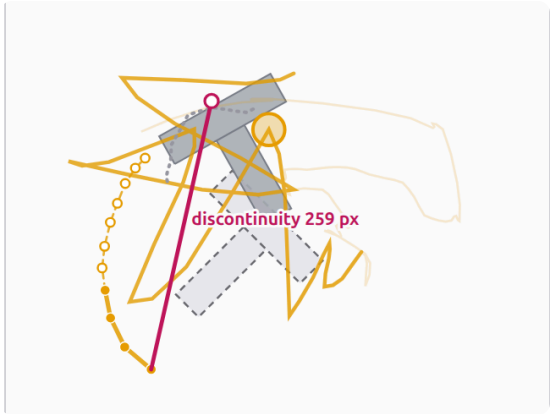

think = policy computing a chunk · act = robot moving (tint changes per chunk) · hatched = idle · lollipop = jump at a switch

## A real case from our data: same start (#1035), d = 4 (400 ms), h = 4

Task: push the grey T-block onto the dashed T-goal (top view). Only the strategy differs.



**Inpaint async** ✓ success




**Naive async** ✗ failure: a 259 px jump at a switch knocks the block away

# Our own recorded runs of a pretrained robot policy

**Task (PushT, simulated, top view)**: a round pusher (the *agent*) must push a T-shaped block onto a fixed T-shaped goal. **Policy**: a pretrained Diffusion Policy (`lerobot/diffusion_pusht`) that turns the last 2 camera frames and pusher positions into a chunk of 16 moves (target positions of the pusher). It builds every chunk from random noise, so running the same schedule again can change an outcome; one *noise replicate* measures how often. Same 50 start states × (46 schedules + 1 noise replicate) = **2,350 episodes** (recorded runs, also called rollouts); every planned chunk and every executed move is logged.

| Attribute (episode table, 2,350 items × 21 attributes) | Type | Order / range / levels | Meaning |
|---|---|---|---|
| **strategy** KEY | nominal | sync · naive · inpaint | what the robot does while a chunk is computed |
| **latency d** KEY | ordinal (ratio, steps) | 0 < 1 < 2 < 4 < 6 steps (0–600 ms) | inference delay; d = 0 only for sync |
| **horizon h** KEY | ordinal (ratio, steps) | 2 < 4 < 6 < 8 | moves executed from each chunk before switching |
| **replicate** KEY | nominal (binary) | 0 · 1 | 1 = sync d0 h8 run again with new random noise (chance level) |
| **seed** KEY | nominal (ID) | 1000–1049 (50 levels) | start state of block & pusher, same in every schedule |
| success | nominal (binary) | 0 = fail, 1 = success | coverage > 95 % within 300 steps |
| coverage_final | quant. ratio, ↑ | 0.00–0.99 | overlap of T-block with goal |
| steps_to_success | quant. ratio, ↓ | 68–300 (empty if failed) | time to finish |
| seam_gap_mean / max | quant. ratio (px), ↓ | 2.3–222.2 / 6–305 | chunk-boundary discontinuity: jump between old and new plan at a hand-over |
| think_motion_mean / max | quant. ratio (px), ↓ | 0.0–106.5 / 0–317 | how far the robot moved while the policy was thinking (change the new chunk did not see) |
| staleness_mean | quant. ratio (steps) | 0.5–9.5 | age of the camera picture behind each move (≈ d + (h−1)/2, equal for all strategies) |
| idle_frac | quant. ratio, ↓ | 0.00–0.75 | share of steps with nothing to execute |
| jerk_mean · path_len · n_inferences | quant. ratio | 1.2–221.3 · 205–12828 px · 8–150 | smoothness (jerk = 2nd difference of actions) · distance travelled · # chunks computed |

**Dataset type**: multidimensional **table** of episodes, each linked to two **temporal** tables and to a **spatial** field (512 × 512 px workspace).

- **Step table** (598,117 items) · keys: schedule, seed, t (ordinal, 10 Hz) · values: agent x, y; block x, y (interval, px); block θ (cyclic, rad); action x, y (interval, px); chunk id (ordinal); index j 0–15 (ordinal); age (ratio, steps since its camera picture); held (binary, 1 = idle step); coverage (ratio 0–1, block–goal overlap).
- **Chunk table** (119,373 items) · keys: schedule, seed, chunk id · values: t_obs (time of the camera picture), t_avail (time the chunk was ready), executed window, 16 predicted (x, y) actions.
- **Size**: 240 KB episode CSV + 112 MB step/chunk JSON (lazy-loaded per schedule).

**Keys vs. values**: keys = (strategy, latency, horizon, replicate) → *schedule*; × seed → episode; × t → step; × chunk id → chunk. Everything else is a value.

Novel data: no public dataset logs per-inference chunks (survey, Oct 2026). Generated by us; scripts in `scripts/`. 1 step = 100 ms.

# Two people who must decide how a policy is executed

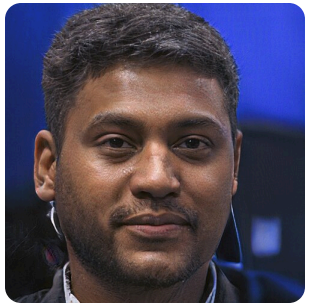


PRIMARY PERSONA

## Mara Jensen

28 · 4th-year PhD student, robot learning lab · Copenhagen, Denmark

- **Context**: trains diffusion / flow-matching policies (generative models that output action chunks); deploys them on a real arm while the GPU server sits across the building (60–250 ms Wi-Fi latency).
- **Goals**: pick an execution strategy and horizon that keep success high at the latency she actually has; explain *why* in a paper.
- **Needs**: per-episode evidence of what failed, aligned with what the policy planned at that moment.
- **Pain points**: only success-rate tables and video; cannot tell whether a failure came from waiting, an outdated plan or a jump; re-running real robot trials is costly; matplotlib debug plots show one episode at a time.



SECONDARY PERSONA

## Arjun Mehta

37 · ML platform engineer, warehouse-robotics start-up · Seoul (Pangyo), Korea

- **Context**: owns the policy-serving software (the robot sends camera frames to a GPU server, which returns action chunks); does not train models.
- **Goals**: set a latency budget (the largest inference delay a robot may tolerate) per robot and a safe default strategy; know when to fall back to sync.
- **Needs**: an overview of how success degrades with latency for each strategy, with confidence that it is not a few lucky seeds.
- **Pain points**: research papers report curves on other robots; no shared vocabulary with researchers about “why it jitters”.

| Difference | Mara (primary) | Arjun (secondary) |
|---|---|---|
| Expertise | Policy internals (diffusion, chunking) | Systems, networking, latency |
| Granularity of questions | Single episodes, single chunk hand-overs | Schedules and aggregate trends |
| Main view used | Workspace replay + execution timeline, seed shifts (T3, T4) | Latency-response curves + paired seed table (T1, T2) |
| Decision made | Which strategy / horizon to propose and why | Latency budget and fallback rule |

Portraits: StyleGAN-generated, public domain (Wikimedia Commons “Woman 1.jpg”, “Man 2.jpg”); the people do not exist. Personas are fictional.

# Four tasks, from all schedules down to a single chunk switch

| # | Action + target | Concrete question (Mara) | Why | Expected outcome |
|---|---|---|---|---|
| **T1** | **Compare → trends** of success and of each strategy's cost (idle time, outdated plans, jumps) across schedules | "How fast does success fall with latency for each strategy, and which horizon is most robust?" | Choose a candidate schedule for the latency the lab actually has. | A shortlist of robust schedules and a latency budget per strategy. |
| **T2** | **Locate → outliers**: initial states whose outcome **flips** between a reference and a candidate schedule | "Going from sync d=0 to naive async d=4, *which* seeds start failing, and which are rescued?" | Aggregate rates hide which situations break; paired seeds isolate the schedule's effect. | A small set of fragile seeds to inspect (and to add to a regression test set). |
| **T3** | **Identify → features** in time & space: where idling, outdated plans and jumps occur in a flipped episode | "At which step does the candidate diverge from the reference, and did the robot wait, act on an outdated plan, or jump?" | Diagnose the mechanism of failure, not just its rate. | A causal hypothesis, e.g. "a 30 px discontinuity right at block contact makes the push slip." |
| **T4** | **Discover → correlation** between jumps at chunk switches (boundary discontinuities) and task outcome (coverage) over paired episodes | "When a seed moves from R to C, do larger discontinuities go with lower coverage? Does inpainting shrink the discontinuities?" | If jumps are what breaks episodes, a strategy that shrinks them should rescue seeds; check it on our policy. | Evidence for or against a strategy (direction and size of shift per seed). |

**Arjun** mainly needs **T1** (latency budget) and **T2** (is the drop driven by a few seeds?). **Mara** chains T1 → T2 → T3 → T4 in one session: overview first, then drill down to a single chunk hand-over (details on demand).

# Each view answers one task with the most effective idiom

| View | Task | Idiom & why (VIS literature) | Interaction |
|---|---|---|---|
| **A · Latency Response** | T1 | *"How fast does each strategy get worse as the policy gets slower?"* **Line chart small multiples**: one panel per horizon h, x = latency d on its true scale, y = metric, strategies **superposed** (hue + shape). Position on a common scale beats the luminance of a heatmap [Cleveland & McGill 84; Heer & Bostock 10]; superposition for direct sync-vs-async comparison, juxtaposition across h [Gleicher 11]. **95 % interval** lines instead of bars [Correll & Gleicher 14]; hover = paired difference to R. | Click mark = C (compare), shift-click = R (baseline); **one question bar**: Success · Idle (cost of sync) · Outdated (cost of async) · Discontinuity (cost of naive async), drives A *and* B; recomputes on seeds selected in B. |
| **B · Paired Seed Table** | T2, T4 | *"Which start states flip between R and C, and is that more than chance?"* **Composite Table-Lens view** [Rao & Card 94; Javed & Elmqvist 12]: one row per seed shared by (left) an outcome matrix — filled = success, R column black, C column orange — and (right) **dumbbells** R → C of the attribute that belongs to A's question. Rows are **grouped by R→C change** (spatial region), so flips break their band's pattern; a **re-run** column shows chance flips (11/50). | No controls of its own. Click column header = C; click cell = open episode; **brush** dumbbells or click a band → **cross-filter** A [Weaver 10]. |
| **C · Workspace Replay** | T3 | *"Watch both runs side by side: what was planned, what was done, where it jumped."* Juxtaposed R \| C maps at identical scale (true geometry). Overlays: current chunk, executed (solid) and thrown away (dashed); next chunk being computed (dotted); latest **discontinuity** (magenta); **what the policy saw** (dotted outline of block & pusher in the chunk's camera picture, arrow to the robot now: the gap = how outdated the plan is). | Play / scrub / ← → (**navigate** t); hover a planned point → when planned / executed; click → jump. |
| **D · Execution Timeline** | T3 | *"When was the policy thinking, which plan did each move come from, where were the jumps?"* **Gantt lanes** on a shared linear time axis [Plaisant 96; Brehmer 17]: per run a **think** track (inference) and an **act** track (chunk spans, hatched = idle), discontinuities as **lollipops** (length), coverage lines below. Sync shows think/act *alternating*, async *overlapping*. The chunk at the cursor is traced **picture ○ → think → act**. | **Brush** a time window → highlighted path in C; hover chunk → its plan in C; click → cursor. |

**Why this is not a generic dashboard**: C and D trace every executed move back to the plan (chunk) and the camera frame it came from, which no chart tool offers; A and B compare two schedules *start state by start state* against a chance baseline (re-run), not just averages. **Less is more**: the first draft had 5 views and ~10 hues; following parsimony and consistency [Wang Baldonado 00] we merged two views, gave every hue one meaning and cut ten controls to **one question bar**.

# The most important attribute gets the most effective channel

| Attribute (type) | View | Mark | Channel | Justification |
|---|---|---|---|---|
| success rate, idle share, motion while thinking, boundary discontinuity (ratio) | A | point + line | position y (common scale) | best magnitude channel; slope = degradation |
| latency d (ratio) | A | — | position x (true spacing) | 100 ms steps, not ordinal bins |
| horizon h (ordinal) | A | panel | spatial region (small multiples) | juxtaposed, shared y |
| strategy (nominal, 3) | A, B | line, point | hue + shape (●■◆) | redundant: never hue alone |
| uncertainty (95 % interval) | A | thin line | position (extent) | avoids within-the-bar bias |
| success per seed (binary) | B | cell | luminance (filled / empty) | 2 levels → 1 luminance step |
| R→C change (nominal, 4) | B | row band | spatial region | top categorical channel |
| attribute under R and C (ratio) | B | dumbbell | position x (symlog if long-tailed) | aligned shift, no crossing |
| agent / block / plan (interval, px) | C | line, point, polygon | position (x, y) | spatial data, true geometry |
| executed / discarded / computing | C | line, point | line style + fill | separable, keeps hue for identity |
| think, act intervals (time) | D | bar | position + length on time axis | alternation vs overlap visible |
| boundary discontinuity (ratio, px) | C, D | lollipop, segment | length + magenta hue | magnitude → length |
| coverage (ratio) | D | line | position y | R vs C on a common scale |
| scene in the picture vs. now (interval, px) | C | dotted outline, arrow | position (x, y), same map | gap = outdated plan, read in place |
| chunk provenance (picture → think → act) | D | ○ + outlines + link | position on time axis | connects one chunk's three moments |

**Hue budget — one meaning per hue**

■ Reference R ■ Candidate C

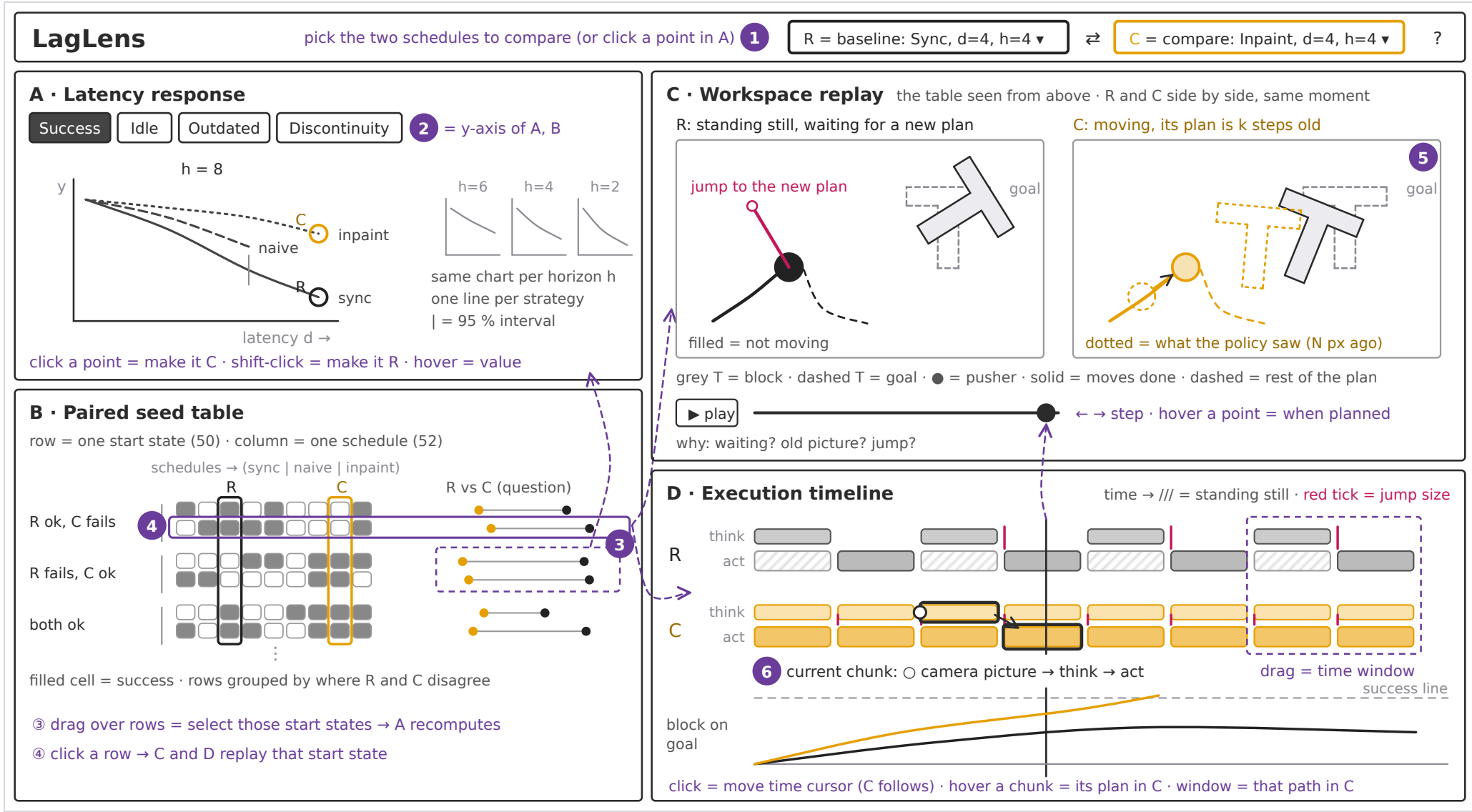
● Sync ■ Naive async ◆ Inpaint

■ Discontinuity (C, D) grey: everything else

Okabe–Ito hues. CVD-checked with Machado et al. (VIS 2009) in OKLab: R vs C ΔE ≥ 48; strategy hues ≥ 9 plus redundant shape.

**Ranking used** (Munzner, Ch. 5):
magnitude: position on common scale > length > angle/area > luminance
identity: spatial region > hue > shape

**Scalability**: A 47 points · B 2,350 cells + 50 dumbbells · C/D one episode (≤ 300 steps, ≤ 150 chunks) — SVG at interactive rates.




1. **Pick R and C**: in the header, or click / shift-click a mark in A.
2. **Choose the question** (Success · Idle · Outdated · Discontinuity): A's y-axis and B's dumbbells follow.
3. **Find flipped start states** in B; drag over the dumbbells to select some, and A recomputes on them.
4. **Click a start state**: C replays it under R and C side by side (▶ play, ← →).
5. **See why** in C: what the policy saw (dotted) vs. where the robot is now, and the jump at each chunk switch.
6. **See when** in D: the current chunk is traced picture → think → act; drag a window to highlight its path in C.

Low-fidelity sketch: marks are schematic placeholders, not data. d3.js v7 only.